



Phantom PJ Empty Leg Alerts

Technical documentation for the Telegram to Slack alert bridge:
architecture, server layout, external service configuration,
deployment and operations.

Repository github.com/phantomstay/tg-alert-slack · public
Host tg-alert.phantomstay.com · 159.89.92.61 · DigitalOcean NYC1
Destination Slack #flight-pj-alerts
Revised 3 September 2026

Contents

- 1 **System overview** — what it does and why it works this way
- 2 **How a message becomes a card** — parsing, filtering, dedupe, pacing
- 3 **Server anatomy** — accounts, paths, privilege boundaries
- 4 **External services** — GoDaddy, Slack, Resend, DigitalOcean, Telegram, GitHub
- 5 **Deployment** — the pipeline and why it pulls rather than pushes
- 6 **Monitoring** — health checks and where failures go
- 7 **Configuration reference** — every environment variable
- 8 **Operations** — day to day commands
- 9 **Security model**
- 10 **Known limits and open items**

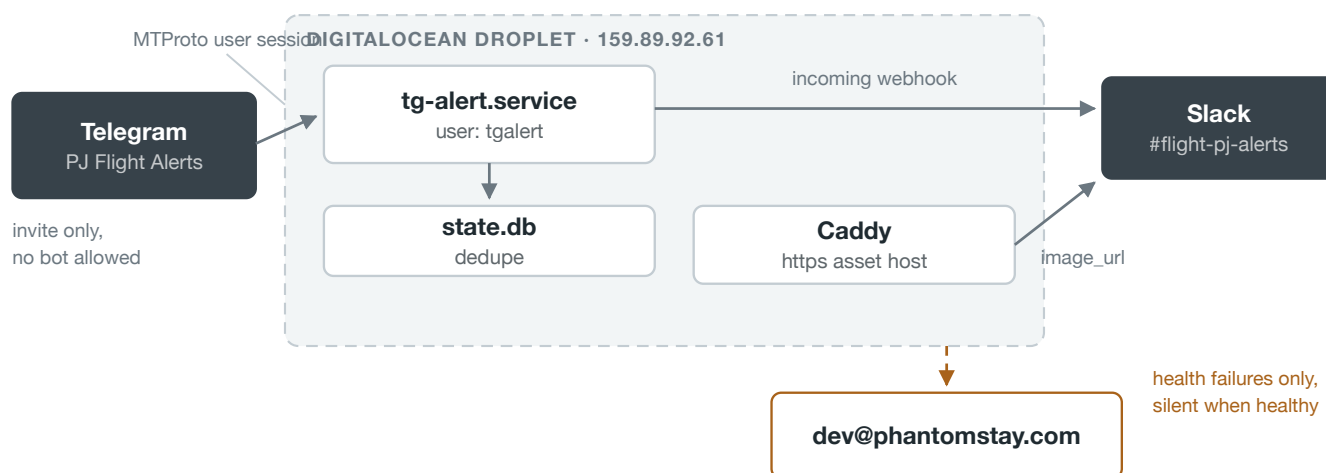
The short version

A private Telegram channel posts private jet empty leg deals. This service reads that channel with a logged-in user account, turns each post into a formatted Slack card with a photo of the aircraft, and posts it into `#flight-pj-alerts` within a second or two of it appearing.

It runs continuously on a small DigitalOcean droplet, deploys itself when you push to `main`, and emails a named address when any part of it breaks. It does not email anyone when things are working.

Why not a Telegram bot. A bot cannot read a channel unless the channel owner promotes it to admin. This channel is invite-only, the owner is not our client, and they declined. The only remaining route is a real user session over MTPROTO. That has a consequence that shapes everything else: the session token can read *everything* on the account it belongs to, so it is treated as a high-value secret throughout.

1 System overview



The whole system. Solid lines carry flight alerts, the dashed line carries failure notices and is silent when everything works.

Four moving parts, all on one host.

tg-alert.service

A long-running Python process holding an authenticated MTPROTO connection to Telegram. It receives new channel messages as events, parses each one, and posts the ones that are flight alerts to Slack. It restarts automatically on failure and catches up on anything it missed while down.

state.db

SQLite, one table, one row per message id already posted. This is what makes restarts, catch-up and manual backfills safe to repeat: the same alert cannot reach Slack twice.

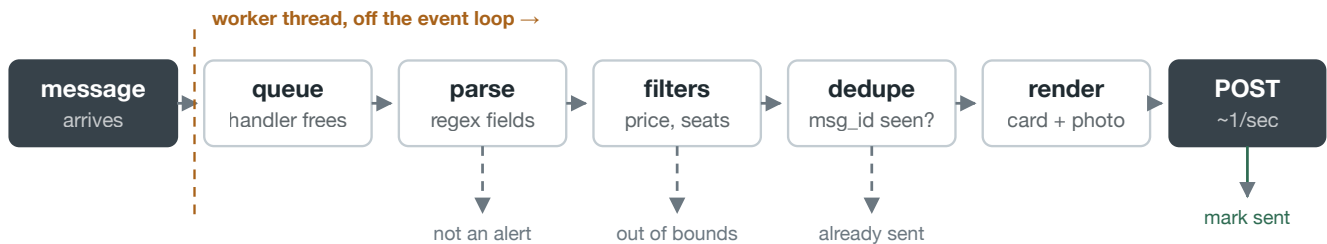
Caddy

Serves the aircraft photographs over HTTPS with an automatically renewed Let's Encrypt certificate. This exists because of a Slack constraint: an `image_url` in a Block Kit card is fetched by *Slack's own servers*, not the browser. A local file path or a Slack-hosted file will not render. The image must be publicly reachable over HTTPS.

Resend

Delivers health failure emails over HTTPS. Covered in section 6, along with the reason SMTP is not an option on this host.

2 How a message becomes a card



A message runs this gauntlet in a worker thread. Three gates can discard it before it reaches Slack.

Parsing

Alerts follow a fixed shape, so `alerts.py` pulls named fields out with regular expressions: route, date, aircraft type, seats and price. Anything that does not match the shape is not an alert and is ignored, which is how ordinary chatter in the channel is filtered out for free.

Choosing the photo

`aircraft_images.json` maps aircraft name fragments to image URLs, and the *longest* matching fragment wins, so `global 7500` beats a generic `global` entry. If nothing matches, `ALERT_IMAGE_URL` is used. If that is unset, the card posts without a photo rather than failing.

A bad image URL kills the whole card. Slack rejects the entire message with `invalid_blocks` if it cannot fetch the image, so an unreachable URL means no alert at all, not an alert without a picture. The health check verifies the image host returns `200` and an image content type for exactly this reason.

Why the work happens on a thread

Posting to Slack is blocking and deliberately paced at about one message per second, because incoming webhooks rate limit above that. Doing that inline inside the Telegram event handler would stall Telethon's event loop for the entire length of a bulk drop. Twenty-five alerts arriving at once would freeze the connection for half a minute, long enough for Telegram to drop it.

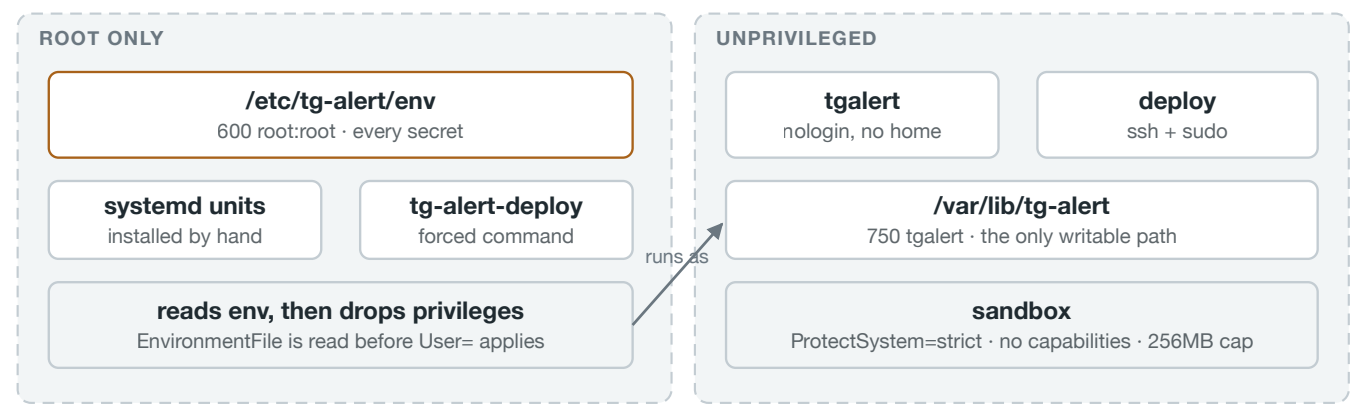
So the handler does one thing: put the message on a queue and return. A single worker thread drains that queue. Order is preserved, the connection stays responsive, and a burst of any size is handled without risking the session.

Bulk drops

Every alert in a burst is posted. There is no cap, no batching and no summarising. Twenty five alerts means twenty five cards over roughly thirty seconds.

3 Server anatomy

Ubuntu 24.04 LTS, 1 vCPU, 458 MB of RAM and a 1 GB swap file, in DigitalOcean's NYC1 region. The region is not arbitrary: a user session connecting from a datacenter in a different country to the account's phone number is exactly what Telegram's anti-fraud systems look for.



Three privilege domains. Secrets are readable only by root, and the process that uses them never runs as root.

Accounts

Account	Shell	Purpose
deploy	/bin/bash	the only account permitted to log in over SSH, has sudo
tgalert	nologin	runs the service, no home directory, cannot log in
caddy	nologin	runs the web server
root	—	direct login disabled entirely

Paths

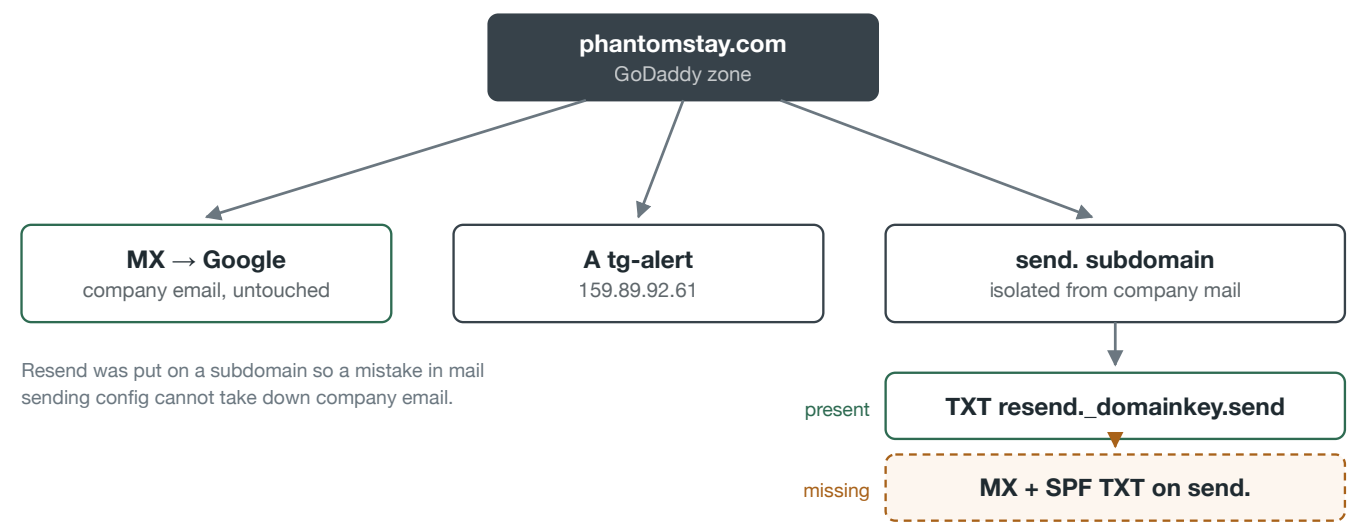
Path	Owner	Mode	Holds
/opt/tg-alert	deploy	755	the git checkout and virtualenv
/etc/tg-alert/env	root	600	every secret
/var/lib/tg-alert	tgalert	750	state.db, the only writable path
/var/www/tg-alert	root	755	aircraft photos served by Caddy
/usr/local/bin/tg-alert	root	755	the operator wrapper
/usr/local/bin/tg-alert-deploy	root	755	the only command CI can run

How secrets reach an unprivileged process. systemd reads `EnvironmentFile` as root, *before* applying `User=tgalert`. So the env file stays `600 root:root` and unreadable by the service account, while the service still receives its contents. The `tgalert` user cannot read its own credentials from disk.

4 External services

Six accounts outside the repository hold state this project depends on.

GoDaddy DNS



The `phantomstay.com` zone. Company email lives at the apex and was left alone; everything added for this project sits on separate names.

Type	Host	Value	Why it exists
A	tg-alert	159.89.92.61	Slack fetches card images from a public host, so the photos need a real HTTPS address
TXT	resend._domainkey.send	DKIM public key	lets Resend sign mail as the domain, required before it will send to anyone

The apex `MX` records and apex `SPF TXT` belong to Google Workspace and were deliberately not touched. Resend was configured on the `send.` subdomain specifically so that a mistake in mail sending configuration cannot take down company email.

Open item. Resend also asks for an `MX` record and an `SPF TXT` on `send.phantomstay.com`. Neither is present, verified directly against GoDaddy's authoritative nameserver. Mail sends today because DKIM alone is enough to authorise it, but without SPF alignment deliverability is weaker and bounces are not tracked. Adding both is low risk, they only affect the subdomain.

Slack

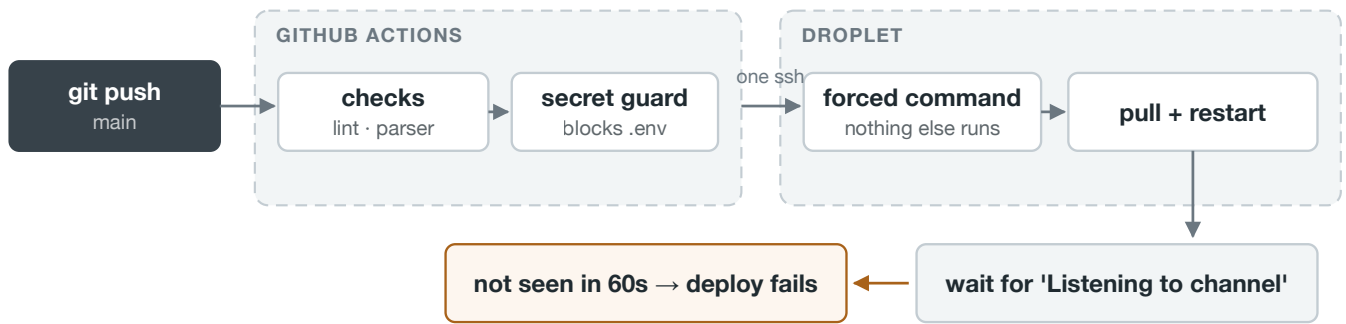
- App **Phantom PJ Empty Leg Alerts**. Twenty-seven characters, because Slack caps app names at thirty-five
- Icon 1024×1024 PNG, background colour `#0c212c`
- One **incoming webhook** bound to `#flight-pj-alerts`, scope `incoming-webhook`
- Installation required workspace admin approval; the developer account is not an admin

The webhook URL is simultaneously the credential and the channel binding. Anyone holding it can post to that channel, and it cannot be pointed elsewhere. Changing channel means issuing a new webhook. It lives only in `/etc/tg-alert/env`.

Resend, DigitalOcean, Telegram, GitHub

Service	What is configured
Resend	domain <code>send.phantomstay.com</code> , DKIM verified, API key in the server env file
DigitalOcean	the droplet; outbound ports 25, 465 and 587 are blocked at the platform level and cannot be opened from inside
Telegram	<code>api_id</code> and <code>api_hash</code> from my.telegram.org, registered against the account holding the session and not portable to another account. Access is revocable at any time under Settings → Devices
GitHub	public repository. Four secrets drive deployment: <code>SSH_HOST</code> , <code>SSH_USER</code> , <code>SSH_PRIVATE_KEY</code> , <code>SSH_KNOWN_HOSTS</code>

5 Deployment



Push to main and this runs. The deploy fails loudly rather than leaving a dead service behind.

Continuous integration lints the code, runs an assertion test that parses a real PJX-shaped alert and checks the resulting card, and fails the build if a tracked `.env` file or a committed webhook URL is found. Only then does it deploy.

Why it pulls instead of pushing

CI never copies files to the server. It opens one SSH connection and sends no command at all. The key it uses is pinned in `authorized_keys` to a single forced command, `tg-alert-deploy`, which pulls `main`, restarts the service, and waits up to sixty seconds for the log line confirming it is listening. If that line does not appear, the script exits non-zero and the deploy is marked failed.

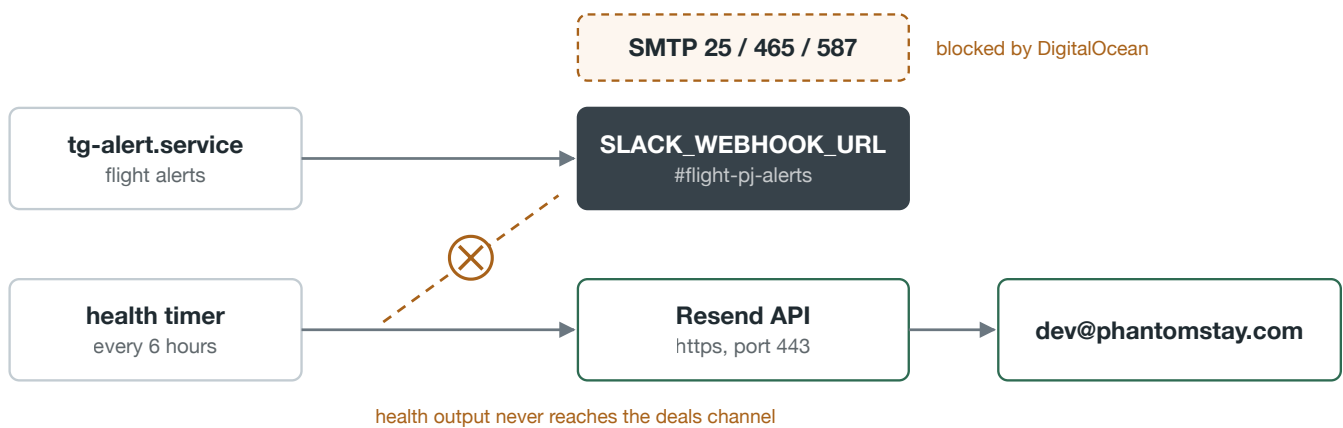
The consequence is that a leaked deploy key can do exactly one thing: deploy the current contents of `main`. It cannot read the environment file, open a shell, or copy files off the box. This was verified by attempting all four.

systemd units are installed by hand, on purpose. They are kept in the repository for reference but the deploy script never touches them. If deployment installed unit files, then push access to a public GitHub repository would be a path to running arbitrary code as root.

Host key pinning

`SSH_KNOWN_HOSTS` pins the server's host key in CI, so a hijacked DNS record cannot cause the workflow to hand its private key to a different machine.

6 Monitoring



Two destinations, deliberately kept apart. Health output cannot reach the channel the client reads.

A systemd timer runs a health check every six hours, and again ten minutes after boot. It checks six things and prints a line for each.

```
telegram session OK    logged in as Alex (id 8448749313)
channel access  OK    'PJ Flight Alerts', latest post id 390
feed activity   OK    last post 51.1h ago
slack webhook   OK    400 invalid_payload (reachable)
alert image     OK    200 image/png .../assets/global-express.png
dedupe db       OK    /var/lib/tg-alert/state.db, 40 alerts recorded
```

The check that matters

Five of these fail loudly on their own. **Feed activity** is the one that earns its keep. A session revoked by Telegram, a channel that quietly stops delivering, or an account locked out looks identical from the outside to a genuinely quiet week. Without this check the service could sit there, healthy in every measurable way, delivering nothing, indefinitely.

`STALE_HOURS` controls the threshold. It ships at 48 and production runs 96, because this channel routinely goes two days without posting.

Why mail goes over HTTPS

DigitalOcean blocks outbound SMTP on this droplet. Every provider times out:

```
smtp.gmail.com:587      TimeoutError
smtp.resend.com:587     TimeoutError
smtp-relay.brevo.com:587 TimeoutError
api.resend.com:443      OK
```

The error message is misleading. The failure surfaces as `[Errno 101] Network is unreachable`, which points at IPv6. That is real, the droplet has no IPv6 route, but it is not the problem. Python tries the IPv6 address last, so its error is the one that surfaces, hiding the IPv4 timeout underneath. Anyone debugging this from the error text alone will spend an hour on the wrong thing.

Mail therefore goes through the Resend API over port 443. The SMTP settings remain in the code and work on any host that permits outbound SMTP.

Notification discipline

Repeat notices are suppressed for `NOTIFY_COOLDOWN_HOURS`, twelve by default, so a week-long outage produces one email rather than twenty-eight. Health output never goes to `SLACK_WEBHOOK_URL`; there is no fallback path to it. Ops noise in a deals channel teaches people to scroll past the channel.

7 Configuration reference

All configuration is environment variables, in `/etc/tg-alert/env` on the server and `.env` locally. The template is `.env.example`.

TELEGRAM

Key	Default	Meaning
TG_API_ID TG_API_HASH	required	from my.telegram.org, tied to one account
TG_SESSION	required	Telethon StringSession, full account access
TG_CHANNEL_ID	required	negative id of the source channel

SLACK AND PRESENTATION

Key	Default	Meaning
SLACK_WEBHOOK_URL	required	one webhook posts to one channel
ALERT_IMAGE_URL	none	fallback photo when no aircraft name matches
MAX_PRICE MIN_SEATS	none	drop alerts outside these bounds

STATE AND RECOVERY

Key	Default	Meaning
STATE_DB	state.db	absolute path under systemd
CATCHUP	20	posts read on a first ever run
CATCHUP_MAX	200	ceiling when resuming from the last posted id

HEALTH AND ALERTING

Key	Default	Meaning
STALE_HOURS	48	how long the channel may be quiet before that counts as a failure
NOTIFY_COOLDOWN_HOURS	12	minimum gap between repeat notices
OPS_EMAIL_TO	none	where health failures go
OPS_EMAIL_FROM	SMTP_USER	must be on a verified domain
RESEND_API_KEY	none	send over HTTPS, required on DigitalOcean
SMTP_HOST SMTP_PORT SMTP_USER SMTP_PASS	none	only usable on a host that permits outbound SMTP
SLACK_OPS_WEBHOOK_URL	none	alternative to email, a separate channel

8 Operations

Everything runs through the `tg-alert` wrapper, which re-execs under the service account with the production environment loaded. Running `run.py` directly will not have the credentials.

Checking on it

```
ssh deploy@tg-alert.phantomstay.com

sudo tg-alert --health          # every dependency, as a table
sudo systemctl status tg-alert # is it running
sudo journalctl -u tg-alert -f  # live log
systemctl list-timers tg-alert-health.timer
```

Proving the chain works

```
sudo tg-alert --test-post  # one card marked "test, not a real flight"
sudo tg-alert --test-notify # one email to the ops address
```

`--test-post` exercises Telegram authentication, parsing, the image host and the Slack webhook in a single command. `--test-notify` ignores the cooldown and runs before the Telegram client is even constructed, so you can still test your alerting when the Telegram session is the thing that is broken.

Changing the Slack channel

Create a new incoming webhook for the target channel in the Slack app, then:

```
sudo nano /etc/tg-alert/env      # replace SLACK_WEBHOOK_URL
sudo systemctl restart tg-alert
sudo tg-alert --test-post
```

The old webhook keeps working until revoked, so both channels can run during a switchover. If the new channel is private, the app must be invited to it before a webhook can be created.

Seeding a channel with recent alerts

```
sudo tg-alert --backfill 20 --send --force
```

`--force` is required because setup ran `--mark-seen`, which put existing message ids in the dedupe database. Without it, backfill prints "already sent, skipping" and posts nothing. Backfill posts oldest first so the channel reads in real order, and paces itself to stay under the webhook rate limit.

Adding an aircraft photo

Drop the image in `assets/`, add a lowercase name fragment and its public URL to `aircraft_images.json`, push, then copy the file to `/var/www/tg-alert/assets/` on the server. The longest matching fragment wins.

9 Security model

Access to the host

- Root login disabled, password authentication disabled, `deploy` is the only permitted SSH user
- `ufw`: port 22 rate limited, 80 and 443 open, everything else denied inbound
- `fail2ban` watching the `sshd` journal
- Unattended security upgrades enabled

The hardening was sequenced so logout was impossible: the `deploy` account was created and its login *and* `sudo` verified working before root was disabled, and port 22 was allowed in `ufw` before the firewall was enabled.

Containment of the service

- Runs as `tgalert`, which has no shell and no home directory
- `ProtectSystem=strict`, an empty capability bounding set, a `syscall` filter, and `ReadWritePaths` limited to `/var/lib/tg-alert`
- Capped at 256 MB, on a host with 458 MB, so a leak cannot take the box down

Secrets

- Nothing sensitive is in the repository, which is public
- `.gitignore` excludes `.env.*`, not just `.env`
- CI fails the build if a tracked `env` file or a committed webhook URL appears
- The deploy key is locked to a single forced command

Standing risk outside this system. API keys for this project's wider programme are pinned in a Slack channel readable by external vendors. The Telegram session token must never be posted there. It grants full read access to the account, not just to this one channel.

10 Known limits and open items

Item	Detail
Missing SPF and MX on <code>send.</code>	mail sends on DKIM alone; adding them improves deliverability and enables bounce tracking
No cap on bulk drops	every alert in a burst posts, about one per second, with no summarising
Backfilled alerts look live	they are real past deals and carry no expiry marker
One webhook, one channel	changing destination means issuing a new webhook
Resend sandbox sender	<code>onboarding@resend.dev</code> only delivers to the Resend account owner; a verified domain is needed to send elsewhere
Ops email goes to a personal address	should become a shared inbox at handover; one line in the <code>env</code> file

Single host

no redundancy. The droplet is the whole system